

Master program
" Advanced Analytics for Business"

Advanced Visual Text Analytics – Natural Language Processing



Universitatea Națională de Știință și Tehnologie Politehnica București
Facultatea de Automatică și Calculatoare



Course 5
Autocorect



Lecture content

- Autocorrect
 - Definition.
 - Building the model.
 - Minimum edit distance definition.
 - Minimum edit distance algorithm.
- Python functions
 - Process data.
 - Get count for a specific word.
 - Get probability for a specific word.
 - Strings manipulation using list comprehensions (delete, insert, replace).
 - Letter edits.
 - Spelling suggestions.
 - Minimum edit distance.

Autocorrect

- Function that automatically makes or suggests corrections for mistakes in spelling or grammar made while typing.



- NLP algorithms are trained to correct spellings and errors while typing text.
- Algorithm steps:
 - Identify misspelled words.
 - Find 'n' strings edit distance away.
 - Candidates filtering.
 - Calculate probabilities of candidates.

1. *Identify the misspelled word:*

- Check whether the specific word is in vocabulary or not. If not, there is a high probability of a typo.

2. *Find strings n edit distance away:*

- Edit: operation which is performed on a string to transform it into another string.
- Examples:
 - **Insert:** add a letter: 'to' -> 'top', 'two'.
 - **Delete:** remove a letter: 'hat' -> 'ha', 'at', 'ht'.
 - **Switch:** swap 2 adjacent letters: 'eta' -> 'eat', 'tea'.
 - **Replace:** change one letter to another: 'jaw' -> 'jar', 'paw'.

3. *Filter candidates:*

- Take all words generated above and keep only those that make sense.

4. *Calculate word probabilities:*

$$P(w) = \frac{C(w)}{V}$$

Minimum Edit Distance

- Allows to:
 - Evaluate similarity between strings.
 - Find the minimum number of edits between two strings.
 - Implement spelling correction, document similarity, machine translation, etc.

Example:

“p l a y” -> **“s t a y”**

Edit cost:

p -> s: replace (cost = 2)

l -> t: replace (cost = 2)

2 edits -> ***total cost = 4.***

Edit cost:

Insert 1

Delete 1

Replace 2

- Start with source word and transform into target word:

Source: play → Target: stay

Cost: insert: 1, delete: 1, replace: 2

$p \rightarrow s$

insert + delete: $p \rightarrow ps \rightarrow s$: 2

delete + insert: $p \rightarrow \# \rightarrow s$: 2

replace: $p \rightarrow s$: 2

		0	1	2	3	4
		#	s			
0	#	0	1			
1	p	1	2			
2						
3						
4						

- From # to # cost = 0 – no action.
- From **p** to # cost = 1 – insert.

- Generalize:

$$D[i, j] = \min \left\{ \begin{array}{l} D[i-1, j] + del_cost \\ D[i, j-1] + ins_cost \\ D[i-1, j-1] + \begin{cases} rep_cost, & \text{if } src[i] \neq tar[j] \\ 0, & \text{if } src[i] = tar[j] \end{cases} \end{array} \right\}$$

	0	1	2	3	4	
	#	s	t	a	y	
0	#	0	1	2	3	4
1	p	1	2			
2	l	2				
3	a	3				
4	y	4				



$$D[1, 2] = \min \left\{ \begin{array}{l} 2 + 1 \\ 2 + 1 \\ 1 + \begin{cases} 2, & \text{if } src[i] \neq tar[j] \\ 0, & \text{if } src[i] = tar[j] \end{cases} \end{array} \right\}$$

		0	1	2	3	4
		#	s	t	a	y
0	#	0	1	2	3	4
1	p	1	2	3	4	5
2	l	2	3	4	5	6
3	a	3	4	5	4	5
4	y	4	5	6	5	4

DeepLearning.AI

- Generalize:

$$D[i, j] = \min \left\{ \begin{array}{l} D[i-1, j] + del_cost \\ D[i, j-1] + ins_cost \\ D[i-1, j-1] + \begin{cases} rep_cost, & \text{if } src[i] \neq tar[j] \\ 0, & \text{if } src[i] = tar[j] \end{cases} \end{array} \right\}$$

	0	1	2	3	4	
	#	s	t	a	y	
0	#	0	1	2	3	4
1	p	1	2			
2	l	2				
3	a	3				
4	y	4				



Backtrace – reduce the effort of brute force.

		0	1	2	3	4
		#	s	t	a	y
0	#	0	1	2	3	4
1	p	1	2	3	4	5
2	l	2	3	4	5	6
3	a	3	4	5	4	5
4	y	4	5	6	5	4

DeepLearning.AI

Python functions

- Connect to data, data preprocessing:
 - Find a trustworthy and reliable source data (e.g. Kaggle, <https://odsc.medium.com/20-open-datasets-for-natural-language-processing-538fbfaf8e38>)

```
import re
from collections import Counter
import numpy as np
import pandas as pd
```

```
def process_data(file_name):
    """
    Input:
        A file_name.
    Output:
        words: a list containing all the words in the corpus in lower case.
    """
    words = [] # return this variable correctly
    #Open the file, read its contents into a string variable, converting all words to lower case
    with open(file_name, "r") as f:
        text = f.read().lower()
    #Convert every word to lower case and return them in a list.
    words = re.findall(r'\w+', text)

    return words
```

Python functions

- Get count for each word in corpus

```
def get_count(word_l):  
    '''  
    Input:  
        word_l: a set of words representing the corpus.  
    Output:  
        word_count_dict: key is the word and value is its frequency.  
    '''  
  
    word_count_dict = {} # fill this with word counts  
    word_count_dict = Counter(word_l)  
  
    return word_count_dict
```

```
word_count_dict = get_count(word_l)  
print(f"Word count dictionary {(word_count_dict)}")
```

```
Word count dictionary Counter({'as': 3, 'you': 2, 'have': 2, 'some': 2, 'something': 1, 'yet': 1, 'can': 1, 'not': 1, 'come': 1, 'further': 1})
```

Python functions

- Get probabilities for each word in

```
def get_probs(word_count_dict):  
    '''  
    Input:  
        word_count_dict: key is the word and value is its frequency.  
    Output:  
        probs: A dictionary where keys are the words and the values are the probability that a word will occur.  
    '''  
    probs = {} # return this variable correctly  
  
    # get the total count of words  
    V = sum(word_count_dict.values())  
    for k in word_count_dict:  
        probs[k] = word_count_dict[k] / V  
    return probs
```

```
probs = get_probs(word_count_dict)  
print(f"Length of probs is {len(probs)}")  
print(f"P('as') is {probs['as']:.4f}")
```

```
Length of probs is 10  
P('as') is 0.2000
```

Python functions

- Edit: Delete letter

```
[25]: def delete_letter(word, verbose=False):  
    ...  
    Input:  
        word: the string/word for which you will generate all possible words  
              in the vocabulary which have 1 missing character  
    Output:  
        delete_l: a list of all possible strings obtained by deleting 1 character from word  
    ...  
  
    delete_l = []  
    split_l = []  
  
    split_l = [(word[:i], word[i:]) for i in range(len(word))]  
    delete_l = [l + r[1:] for l, r in split_l if r]  
  
    if verbose: print(f"input word {word}, \nsplit_l = {split_l}, \ndelete_l = {delete_l}")  
  
    return delete_l  
  
[31]: delete_word_l = delete_letter(word="dance",  
                                   verbose=True)  
  
input word dance,  
split_l = [('', 'dance'), ('d', 'ance'), ('da', 'nce'), ('dan', 'ce'), ('danc', 'e')],  
delete_l = ['ance', 'dnce', 'dace', 'dane', 'danc']
```

Python functions

- Edit: Switch letter

```
def switch_letter(word, verbose=False):  
    """  
    Input:  
        word: input string  
    Output:  
        switches: a list of all possible strings with one adjacent charater switched  
    """  
  
    switch_l = []  
    split_l = []  
  
    split_l =  
    switch_l =  
  
    if verbose: print(f"Input word = {word} \n split_l = {split_l} \n switch_l = {switch_l}")  
  
    return switch_l
```

```
switch_word_l = switch_letter(word="talk",  
                               verbose=True)
```

```
Input word = talk  
split_l = [('', 'talk'), ('t', 'alk'), ('ta', 'lk'), ('tal', 'k')]  
switch_l = ['atlk', 'tlak', 'takl']
```

Python functions

- Edit: replace letter

```
def replace_letter(word, verbose=False):
    """
    Input:
        word: the input string/word
    Output:
        replaces: a list of all possible strings where we replaced one letter from the original word.
    """

    letters = 'abcdefghijklmnopqrstuvwxyz'

    replace_l = []
    split_l = []

    split_l = ..
    replace_l = ..

    # Sort the list, for easier viewing
    replace_l = sorted(replace_l)

    if verbose: print(f"Input word = {word} \nsplit_l = {split_l} \nreplace_l {replace_l}")

    return replace_l
```

```
replace_l = replace_letter(word='put',
                           verbose=True)
```

```
Input word = put
split_l = [('', 'put'), ('p', 'ut'), ('pu', 't')]
replace_l ['aut', 'but', 'cut', 'dut', 'eut', 'fut', 'gut', 'hut', 'iut', 'jut', 'kut', 'lut', 'mut', 'nut', 'out', 'pat', 'pbt', 'pct', 'p
dt', 'pet', 'pft', 'pgt', 'pht', 'pit', 'pjt', 'pkt', 'plt', 'pmt', 'pnt', 'pot', 'ppt', 'pqt', 'prt', 'pst', 'ptt', 'pua', 'pub', 'puc',
'pud', 'pue', 'puf', 'pug', 'puh', 'pui', 'puj', 'puk', 'pul', 'pum', 'pun', 'puo', 'pup', 'puq', 'pur', 'pus', 'puu', 'puv', 'puw', 'pux',
'puy', 'puz', 'pvt', 'pwt', 'pxt', 'pyt', 'pzt', 'qut', 'rut', 'sut', 'tut', 'uut', 'vut', 'wut', 'xut', 'yut', 'zut']
```

Python functions

- Edit: insert letter

```
def insert_letter(word, verbose=False):  
    '''  
    Input:  
        word: the input string/word  
    Output:  
        inserts: a set of all possible strings with one new letter inserted at every offset  
    '''  
    letters = 'abcdefghijklmnopqrstuvwxyz'  
    insert_l = []  
    split_l = []  
  
    split_l = ..  
    insert_l = ..  
  
    if verbose: print(f"Input word {word} \nsplit_l = {split_l} \ninsert_l = {insert_l}")  
  
    return insert_l
```

```
insert_l = insert_letter('put', True)  
print(f"Number of strings output by insert_letter('at') is {len(insert_l)}")
```

```
Input word put  
split_l = [('', 'put'), ('p', 'ut'), ('pu', 't'), ('put', '')]  
insert_l = ['aput', 'bput', 'cput', 'dput', 'eput', 'fput', 'gput', 'hput', 'iput', 'jput', 'kput', 'lput', 'mput', 'nput', 'oput', 'pput',  
'qput', 'rput', 'sput', 'tput', 'uput', 'vput', 'wput', 'xput', 'yput', 'zput', 'paut', 'pbut', 'pcut', 'pdut', 'peut', 'pfut', 'pgut', 'ph  
ut', 'piut', 'pjut', 'pkut', 'plut', 'pmut', 'pnut', 'pout', 'pput', 'pput', 'prut', 'psut', 'ptut', 'puut', 'pvut', 'pwut', 'pxut', 'pyu  
t', 'pzut', 'puat', 'pubt', 'puct', 'pudt', 'puet', 'puft', 'pugt', 'puht', 'puit', 'pujt', 'pukt', 'pult', 'pumt', 'punt', 'puot', 'pupt',  
'puqt', 'purt', 'pust', 'putt', 'puut', 'puvt', 'puwt', 'puxt', 'puyt', 'puzt', 'puta', 'putb', 'putc', 'putd', 'pute', 'putf', 'putg', 'pu  
th', 'puti', 'putj', 'putk', 'putl', 'putm', 'putn', 'puto', 'putp', 'putq', 'putr', 'puts', 'putt', 'putu', 'putv', 'putw', 'putx', 'put  
y', 'putz']  
Number of strings output by insert_letter('at') is 104
```

Python functions

- Generate all words that are **1** edit away

```
def edit_one_letter(word, allow_switches = True):
    """
    Input:
        word: the string/word for which we will generate all possible words that are one edit away.
    Output:
        edit_one_set: a set of words with one possible edit.
    """

    edit_one_set = set()

    edit_one_set.update(delete_letter(word))
    if allow_switches:
        edit_one_set.update(switch_letter(word))
    edit_one_set.update(replace_letter(word))
    edit_one_set.update(insert_letter(word))

    # return this as a set and not a list
    return set(edit_one_set)
```

```
tmp_word = "to"
tmp_edit_one_set = edit_one_letter(tmp_word)
# turn this into a list to sort it, in order to view it
tmp_edit_one_l = sorted(list(tmp_edit_one_set))
print(f"input word {tmp_word} \nedit_one_l \n{tmp_edit_one_l}\n")
```

```
input word to
edit_one_l
['ao', 'ato', 'bo', 'bto', 'co', 'cto', 'do', 'dto', 'eo', 'eto', 'fo', 'fto', 'go', 'gto', 'ho', 'hto', 'io', 'ito', 'jo', 'jto', 'ko', 'k
to', 'lo', 'lto', 'mo', 'mto', 'no', 'nto', 'o', 'oo', 'ot', 'oto', 'po', 'pto', 'qo', 'qto', 'ro', 'rto', 'so', 'sto', 't', 'ta', 'tao',
'tb', 'tbo', 'tc', 'tco', 'td', 'tdo', 'te', 'teo', 'tf', 'tfo', 'tg', 'tgo', 'th', 'tho', 'ti', 'tio', 'tj', 'tjo', 'tk', 'tko', 'tl', 'tl
o', 'tm', 'tmo', 'tn', 'tno', 'toa', 'tob', 'toc', 'tod', 'toe', 'tof', 'tog', 'toh', 'toi', 'toj', 'tok', 'tol', 'tom', 'ton', 'too', 'to
p', 'toq', 'tor', 'tos', 'tot', 'tou', 'tov', 'tow', 'tox', 'toy', 'toz', 'tp', 'tpo', 'tq', 'tqo', 'tr', 'tro', 'ts', 'tso', 'tt', 'tto',
'tu', 'tuo', 'tv', 'tvo', 'tw', 'two', 'tx', 'txo', 'ty', 'tyo', 'tz', 'tzo', 'uo', 'uto', 'vo', 'vto', 'wo', 'wto', 'xo', 'xto', 'yo', 'yt
o', 'zo', 'zto']
```


Python functions

- Generate all words that are 2 edits away

```
def edit_two_letters(word, allow_switches = True):  
    """  
    Input:  
        word: the input string/word  
    Output:  
        edit_two_set: a set of strings with all possible two edits  
    """  
  
    edit_two_set = set()  
  
    edit_one = ..  
    for w in edit_one:  
        if w:  
            edit_two = ..  
            edit_two_set.update(..)  
  
    return set(edit_two_set)
```

Python functions

- Get corrections

```
def get_corrections(word, probs, vocab, n=2, verbose = False):  
    ...  
    Inputs:  
        word: a user entered string  
        probs: a probability dictionary for each word  
        vocab: a set containing all the vocabulary  
        n: number of possible word corrections you want returned in the dictionary  
    Output:  
        n_best: a list of tuples with the most probable n corrected words and their probabilities.  
    ...  
  
    suggestions = []  
    n_best = []  
  
    suggestions = list((word in vocab and word) or edit_one_letter(word).intersection(vocab) or edit_two_letters(word).intersection(vocab))  
    n_best = [[s, probs[s]] for s in list(reversed(suggestions))]  
  
    if verbose: print("entered word = ", word, "\nsuggestions = ", suggestions)  
  
    return n_best
```



Python functions

- Minimum edit distance algorithm – dynamic programming

```
def min_edit_distance(source, target, ins_cost = 1, del_cost = 1, rep_cost = 2):  
    """  
    Inputs:  
        source, target: strings  
        ins_cost: an integer – insert cost  
        del_cost: an integer – delete cost  
        rep_cost: an integer – replace cost  
    Outputs:  
        D: a matrix of len(source)+1 by len(target)+1 containing minimum edit distances  
        med: the minimum edit distance (med) required to convert the source string to the target  
    """  
    # use deletion and insert cost as 1  
    m = len(source)  
    n = len(target)  
    # initialize cost matrix with zeros and dimensions (m+1,n+1)  
    D = np.zeros((m+1, n+1), dtype=int)  
  
    # Fill in column 0, from row 1 to row m, both inclusive  
    for row in range(1,m+1):  
        D[row,0] = D[row - 1, 0] + del_cost  
  
    # Fill in row 0, for all columns from 1 to n, both inclusive  
    for col in range(1,n+1):  
        D[0,col] = D[0, col - 1] + ins_cost  
  
    # Loop through row 1 to row m, both inclusive  
    for row in range(1,m+1):  
        # Loop through column 1 to column n, both inclusive  
        for col in range(1,n+1):  
            # Initialize r_cost to the 'replace' cost that is passed into this function  
            r_cost = ..  
  
            # Check to see if source character at the previous row  
            # matches the target character at the previous column,  
            if source[row - 1] == target[col - 1]:  
                r_cost = 0  
  
            # Refer to the equation calculate for D[i,j] (the minimum of three calculated costs)  
            D[row,col] = min(.. + del_cost, .. + ins_cost, .. + r_cost)  
  
    # Set the minimum edit distance with the cost found at row m, column n  
    med = D[m, n]  
  
    return D, med
```



Python functions

- Put all together:
 - Select one corpora.
 - Process data.
 - Get count & probabilities.
 - Define edits functions (insert, delete, replace, switch).
 - Edit 1 and 2 distance functions.
 - Get corrections.
 - Minimum edit distance algorithm.
 - Test with different strings (try words longer than 4,5 characters).

Good Luck 😊